Data Transformation

It is rare that we get the data in exactly the right form needed. Often, we will need to create some new variables or summaries, rename some variables, or reorder the observations. These tasks can be achieved by the ***dplyr*** package, which is another core member of ***tidyverse***. As an example, we will use the data set ***nycflights13***. Hence, we first upload it

```
>install.packages("nycflights13")

>library(nycflights13)
```

If we open this data set, we will see only the first few rows and all columns that can fit on one screen. This ***data.frame*** is in fact a ***tibble*** – slightly tweaked form of a ***data.frame***. We will talk about ***tibble*** in detail later. We can view the data set

```
>View(flights)
```

Let us introduce some terminology.

Tibble: Tibbles are data frames, but slightly tweaked to work better in ***tidyverse***.

We also use the following to describe the types of variables given in columns:

int: integer

dbl: doubles (real numbers)

chr: character vectors (strings)

dttm: date-time (a date and a time)

lgl: logical

fctr: factor

date: date

Basic Language of dplyr

There are five key ***dplyr*** functions that allow us to solve a vast majority of our data challenges:

- To pick observations by their values, we use ***filter ( )***
- To reorder rows, we use ***arrange ( )***
- To pick variables by their names, we use ***select ( )***
- To create new variables as functions of some of the existing variables, we use ***mutate ( )***
- To collapse many values down to a single number, we use ***summarize ( )***

Each one of these functions can be used by ***group_by ( )*** which changes the scope from the entire data set to a group.

These six functions provide the verbs of the language of ***dlpyr***, which one can think of as the language of data manipulation. In each one of these verbs

- The first argument is always a ***data.frame***
- The subsequent arguments describe what to do with the ***data.frame***, using the variable names (without quotes)
- The result is a new ***data.frame***

1. Filter

Say we want to select all flights on January 1. We then write,

```
>filter (flights, month == 1, day == 1)
```

Then to save this result as a new ***data.frame***, called say, Jan1, we write

```
>Jan1 <- filter (flights, month == 1, day == 1)
```

We can do both of these operations at the same time by wrapping the statement in ( ):

```
>( Jan1 <- filter (flights, month == 1, day == 1))
```

2. Comparisons

R uses the standard notation for comparisons:

Less than: $<$

Greater than: $>$

Less than or equal to: $<=$

Greater than equal to: $>=$

Equal to: $==$

Not equal to: $!=$

Note that $x = 3$ means we assign the value 3 in place of x (i.e. it means $x < -3$).

We should also make the following point: Note that

>sqrt(3)^2 == 3

>FALSE

or

>(1/17)*17 == 1

>FALSE

because of finite precision arithmetic. In other words, sqrt(3) is only an approximation to $\sqrt{3}$. However,

if we use the ***near*** function, we get

>near (sqrt(3)^2, 3)

>TRUE

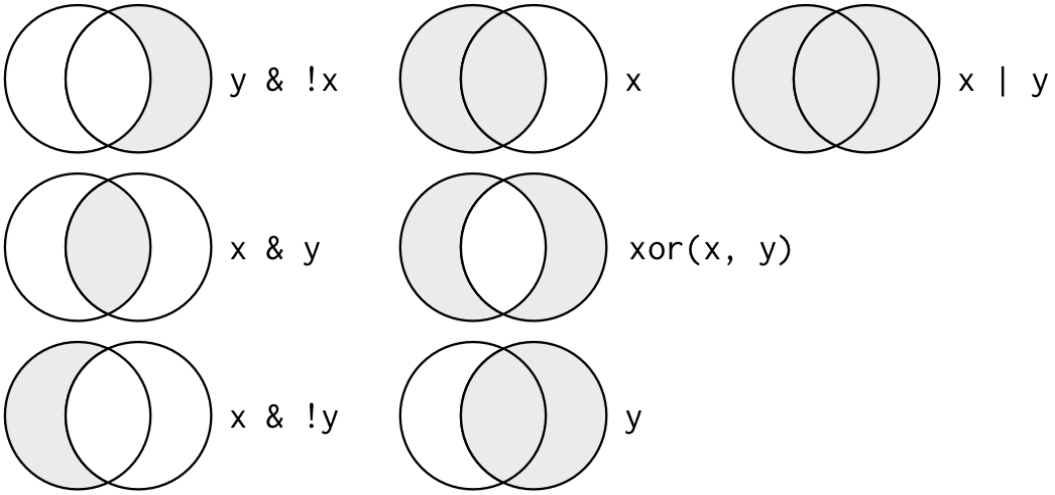
or

>near ((1/17)*17, 1)

>TRUE

3. Logical Operators

If we have multiple arguments to ***filter ( )***, we use the Boolean operators such as “and”, “or”, “not”, and/or their combinations. If expressions are combined with “and”, every expression must be true in order for a row to be included in the output. The following figure shows the complete set of Boolean operations.



Fifth one should be $(x|y)\&\,!\,(x\&y)$

So,

>filter (flights, month == 11 | month == 12)

will list all flights that left on November or December.

Another way this can be written is by ***%in%***:

>filter (flights, month %in% c(11,12))

Some compound statements can be simplified using De Morgan’s Law:

$$!\,(x\,\&\,y) = !\,x\,|\,!\,y$$

and

$$!\,(x\,|y) = !\,x\,\&\,!\,y$$

4. Missing Values

One important feature of R that can make comparison tricky are missing values, or ***NA*** s ("not availables"). ***NA*** represents an unknown value so missing values are "contagious": almost any operation involving an unknown value will also be unknown.

```
>NA > 5
```

> NA

Or

```
>NA == 10
```

> NA

or

```
NA+ 10
```

> NA

or

```
NA/ 2
```

> NA

The most confusing result is this one:

```
>NA== NA
```

> NA

It is easiest to understand why this is true with a bit more context: If the first value is unknown and the second value is unknown, whether they are equal to each other or nor will be unknown.

If we want to determine if a value is missing in a vector x, use ***is.na (x)*** :

>is.na (x)

>TRUE

filter ( ) only includes rows where the condition is **TRUE**; it excludes both **FALSE** and **NA** values. If we want to preserve missing values, we have to ask for them explicitly:

```
>df <-tibble(x =c(1,NA,3))
filter (df, x>1)
```

5. arrange ( )

arrange ( ) works similarly to ***filter ( )*** except that instead of selecting rows, it changes their order, in other words, it orders selected columns. If more than one column name is provided, each additional column will be used to break ties in the values of preceding columns.

By default, the ordering will be ascending. We use ***desc ( )*** to reorder a column in descending order.

```
>arrange (flights, desc (dep_delay) )
```

Missing values are always sorted at the end.

6. select ( )

It is not uncommon to get datasets with hundreds or even thousands of variables. In this case, the first challenge is often narrowing in on the variables one is actually interested in. ***select()*** allows us to rapidly zoom in on a useful subset using the names of the variables.

select() is not terribly useful with data that has relatively few variables such as the flights data which has only 19 variables, but we still get the general idea. For example,

```
>select (flights, year, month, day)
```

will give us just these three columns (variables) in the flights data set.

7. mutate ( )

Besides selecting sets of existing columns, it is often useful to add new columns that are functions of existing columns. That is what ***mutate( )*** does.

mutate( ) always adds the new columns at the end of the data frame, so we will start by creating a narrower dataset so we can see the new variables. Remember that in RStudio, the easiest way to see all the columns is **View( )**.

For example,

>flights_narrow <- select (flights, year : day, ends_with ("delay"), distance, air_time)

>mutate (flights_narrow, gain = dep_delay – arr_delay, speed = (distance/air_time)*60)

Then we will see all the columns of flights_narrow (year, month, day, dep_delay, arr_delay, dist, air_time), plus a column named gain and a column named speed.

If we just want to see the new columns, we use the ***transmute ( )*** function (of course, I this case, we do not need to use the ***select*** function):

>transmute (flights, gain = dep_delay – arr_delay, speed = (distance/air_time)*60)

will just shows the columns gain and speed.

Some Useful Data Creation Functions

There are many functions for creating new variables that we can use with ***mutate ( )***. The key property is that the function must take a vector of values as input, return a vector with the same number of values as output.

There is no way to list every possible function that one might use, but here is a selection of functions that are frequently useful:

Arithmetic operators: +, -, *, /, ^

Modular arithmetic: %/% (integer division) and %% (remainder), where x == y * (x %/% y) + (x %% y). Modular arithmetic is a handy tool because it allows us to break integers up into pieces. For example, in the flights dataset, we can compute hour and minute from dep_time with:

```
transmute(flights,
  dep_time,
  hour = dep_time %/% 60,
  minute = dep_time %% 60
)
```

Logarithms to various bases: log ( ), log2 ( ), log10 ( ). Recall loga (x) stands for $\log_a(x)$. We will talk more about this when we deal with modeling.

All else being equal, we recommend using log2( ) because it is easy to interpret. For, by virtue of the rules

$$\log_a(xy) = \log_a(x) + \log_a(y)$$

$$\log_a(x/y) = \log_a(x) - \log_a(y)$$

$$\log_a(a) = 1$$

a difference of 1 on the $\log_2$ scale corresponds to doubling on the original scale and a difference of -1 corresponds to halving.

Offsets: ***lead (n)*** allow us to refer to elements following the entry n in a vector x and ***lag (n)*** to elements preceding n in a vector x.

```
>x <- 1:10
>lead(x,1)
```

2 3 4 … 9 NA

>lag(x, 1)

NA 1 2 … 9

Cumulative aggregates: R provides functions for running sums, products, minima and maxima: ***cumsum ( )***, ***cumprod( )***, ***cummin( )***, ***cummax( ).*** Moreover, ***dplyr*** provides ***cummean ( )*** for cumulative means.

>x <- 1:10

cumsum (x)

> 1 3 6 10 15 21 28 36 45 55

cummean(x)

> 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0 5.5

Logical comparisons, <, <=, >, >=, !=, ==

Ranking: There are a number of ranking functions, but the most common is ***min_rank( )***. It does the most usual type of ranking (e.g. 1st, 2nd, 3rd, 4th). The default gives smallest values the small ranks; use ***desc(x)*** to give the largest values the smallest ranks.

y <- **c**(1, 2, 2, NA, 3, 4)

min_rank(y)

#> [1] 1 2 2 NA 4 5

min_rank(**desc**(y))

#> [1] 5 3 3 NA 2 1

8. summarize ( )

The last key verb is ***summarize ( )***. It collapses a data frame to a single entity:

>summarize (flights, delay = mean (dep_delay, na.rm = TRUE))

> 12.6

Since ***NA*** is "contagious," if there are any **NA**'s in the vector, the mean will also be ***NA***. Fortunately, all aggregation functions have an argument which removes the missing values prior to computation. Writing

na.rm = TRUE

takes care of this.

summarize ( ) is not terribly useful unless we pair it with ***group_by ( )***. This changes the unit of analysis from the complete dataset to individual groups. When we use the ***dplyr*** verbs on a grouped data frame, they will be automatically applied "by group". For example, if we applied exactly the same code to a data frame grouped by date, we get the average delay per date:

>by_day <- group_by(flights, year, month, day)

>summarize (by_day, delay = mean(dep_delay, na.rm = TRUE))

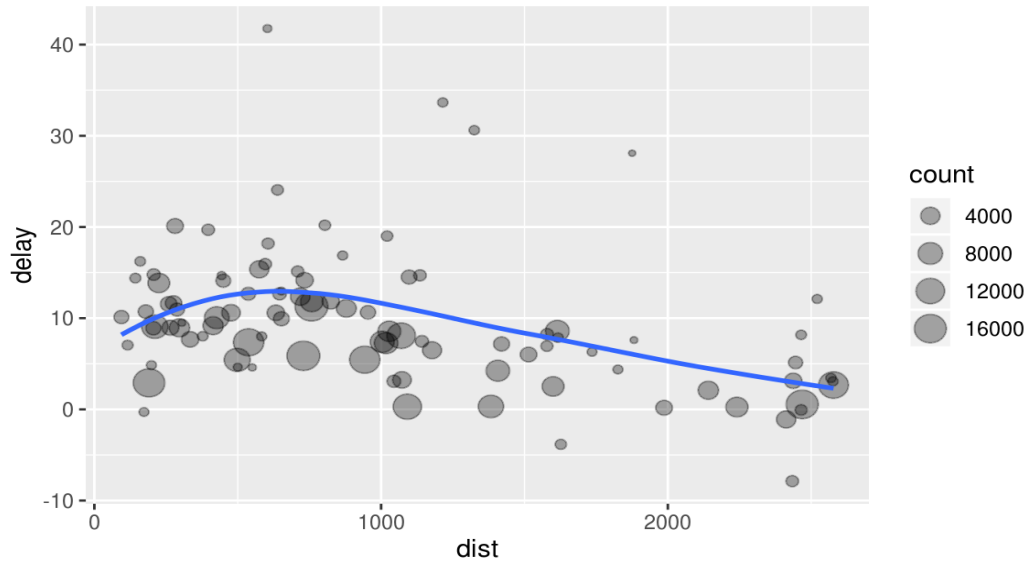
will give us the mean delay for every day of every month. Together ***group_by ( )*** and ***summarize ( )*** provide one of the most commonly used tools in ***dplyr***: ***grouped summaries.***

Imagine that we want to explore the relationship between the distance and average delay for each location. Note that, whenever we do any aggregation, it's always a good idea to include either a count (), or a count of non-missing values (). That way we can check that we are not drawing conclusions based on very small amounts of data.

```
by_dest <- group_by(flights, dest)
delay <- summarize(by_dest,
count = n(),
dist = mean(distance, na.rm = TRUE),
delay = mean(arr_delay, na.rm = TRUE)
)
delay <- filter(delay, count > 20, dest != "HNL")

# It looks like delays increase with distance up to ~750 miles
# and then decrease. Maybe as flights get longer there's more
# ability to make up delays in the air?

ggplot(data = delay, mapping = aes(x = dist, y = delay)) +
geom_point(aes(size = count), alpha = 1/3) +
  geom_smooth(se = FALSE)
#> `geom_smooth()` using method = 'loess' and formula 'y ~ x'
```



There are three steps to prepare this data:

1. Group flights by destination.
2. Summarize to compute distance, average delay, and number of flights.
3. Filter to remove noisy points and Honolulu airport, which is almost twice as far away as the next closest airport.

Useful Summary Functions

1. Measures of central tendency:

   mean (x) and ***median(x)***

   The mean is the sum divided by the length and corresponds to center of mass with masses = 1/(number of elements); the median is a value where 50% of is above it, and 50% is below it.

2. Measures of spread:

   sd(x), ***IQR(x)***, ***mad(x)***

   The root mean squared deviation, or standard deviation, is the standard measure of spread when mean(x) is used as the measure of center. The interquartile range and median absolute

deviation are robust equivalents that may be more useful if we have outliers, i.e., when we use the median as the measure of center.

3. Measures of rank:

   min(x), ***max(x)***, ***quantile (x, r)*** where r is a real number with $0 < r < 1$.

   The ***quantile (x, r)*** gives the number x such that $100r\%$ of the data is below x. So, ***quantile (x,*** $\mathbf{0.25}$***)*** is the first quartile, ***quantile (x,*** $\mathbf{0.5}$***)*** is the median, and quantile *(x,* 0.75*)* is the third quartile.

4. Measures of position:

   first$(\boldsymbol{x})$, ***nth***$(\boldsymbol{x}, \boldsymbol{k})$, ***last***$(\boldsymbol{x})$

   Here, ***nth***$(\boldsymbol{x}, \boldsymbol{k})$ gives the kth component. Hence, these work similarly to $x[k]$ but let us set a default value if that position does not exist (i.e. we are trying to get the 3rd element from a group that only has two elements).

5. Counts:

   $\boldsymbol{n}(\ )$ takes no arguments and returns the size of the current group. We use ***sum (!is.na(x))*** to count the number of non-missing values. To count the number of distinct (unique) values, we use $\boldsymbol{n\_distinct}(\boldsymbol{x})$